

Compressió de dades i imatges - GEI - FIB

Tema 4: Tècniques de diccionari

Anna de Mier, Lluís Vena
anna.de.mier@upc.edu, lluis.vena@upc.edu

Departament de Matemàtiques • Universitat Politècnica de Catalunya

Març 2026

Capítol 5 de [Introduction to Data Compression](#) Sayood, Khalid Morgan
Kaufmann, 2012, 4th ed.

Mètodes de diccionari: generalitats

El 1977 i el 1978 Abraham Lempel i Jacob Ziv van publicar dos articles que introduïen noves idees per a la compressió sense pèrdues

Els seus mètodes es coneixen com a LZ77 i LZ78, i han donat lloc a múltiples variants i millores (LZSS, LZK, LZW...)

Són els que es coneixen com a *mètodes de diccionari*, molt populars per a guardar dades

En particular, s'usen en zip, compress, 7-zip, gif, png, ...

Mètodes de diccionari: generalitats

Ideal general

Construïm un diccionari

Quan el codificador troba una paraula que està al diccionari, es codifica com la referència a l'entrada del diccionari

Aquestes referències les anomenarem *tokens*

Els diferents mètodes difereixen en el contingut del diccionari, en com es construeix (estàtics o adaptatius), en què es guarda als tokens, en com es codifiquen els tokens, etc...

Ziv, Jacob; Lempel, Abraham (May 1977). *A Universal Algorithm for Sequential Data Compression*. IEEE Transactions on Information Theory. 23 (3): 337–343. doi:10.1109/TIT.1977.1055714.

Conegut com el mètode de la *sliding window*

Considerem una finestra de longitud $S + L$; els S primers caràcters són la finestra de cerca i els L darrers la finestra avançada (lookahead buffer) Un token és una tripleta [literal, match-length, offset]=[x, λ, d]
El *match* té longitud $\lambda \leq L$, el símbol x és el primer després del match i el match comença $d \leq S$ posicions enrere des del principi del lookahead buffer

Si no hi ha cap match, es posa $\lambda = d = 0$

Exemple: patadecabraabracadabrapaaaaaaaaaatadecabr

Finestra de cerca: $S = 16$, finestra de text avançat: $L = 8$.

$[x, \lambda, d] = [\text{literal}, \text{match-length}, \text{offset}]$

```

||patadeca|braabracadabrapaaaaaaaaaatadecabr ['p', 0, 0]
|p|atadecab|raabracadabrapaaaaaaaaaatadecabr ['a', 0, 0]
|pa|tadecabr|aabracadabrapaaaaaaaaaatadecabr ['t', 0, 0]
|pat|adecabra|abracadabrapaaaaaaaaaatadecabr ['d', 1, 2]
|patad|ecabraab|racadabrapaaaaaaaaaatadecabr ['e', 0, 0]
|patade|cabraabr|acadabrapaaaaaaaaaatadecabr ['c', 0, 0]
|patadec|abraabra|cadabrapaaaaaaaaaatadecabr ['b', 1, 4]
|patadecab|raabraca|dabrapaaaaaaaaaatadecabr ['r', 0, 0]
|patadecabr|aabracad|abrapaaaaaaaaaatadecabr ['a', 1, 3]
|patadecabraa|bracadab|rapaaaaaaaaaatadecabr ['c', 3, 4]

```

Exemple: patadecabraabracadabrapaaaaaaaaaatadecabr

Finestra de cerca: $S = 16$, finestra de text avançat: $L = 8$.

```
[...]
|patadecabraa|bracadab|rapaaaaaaaaaatadecabr ['c', 3, 4]
|patadecabraabrac|adabrapa|aaaaaaaaaatadecabr ['a', 2, 13]
pat|adecabraabracada|brapaaaa|aaaaaaatadecabr ['p', 3, 7]
patadec|abraabracadabrap|aaaaaaa|aaatadecabr ['a', 2, 13]
patadecabr|aabracadabrapaaa|aaaaaaa|tadecabr ['t', 8, 1]
patadecabraabracada|brapaaaaaaaaaat|adecabr| ['d', 1, 2]
patadecabraabracadabr|apaaaaaaaaaaaatad|ecabr| ['e', 0, 0]
patadecabraabracadabra|paaaaaaaaaaaatade|cabr| ['c', 0, 0]
patadecabraabracadabrap|aaaaaaaaaaaatadec|abr| ['b', 1, 4]
patadecabraabracadabrapaa|aaaaaaaaaatadecab|r| ['r', 0, 0]
['EOF', 0, 0]
```

Codi:

```
['p', 0, 0], ['a', 0, 0], ['t', 0, 0], ['d', 1, 2], ['e', 0, 0], ['c', 0, 0], ['b', 1, 4], ['r', 0, 0], ['a', 1, 3],
['c', 3, 4], ['a', 2, 13], ['p', 3, 7], ['a', 2, 13], ['t', 8, 1], ['d', 1, 2], ['e', 0, 0], ['c', 0, 0], ['b', 1, 4],
['r', 0, 0], ['EOF', 0, 0]
```

- Els valors de S i L afecten a:
 - ▶ la ràtio de compressió: bits a desar *vs* possibilitat de trobar cadenes més llargues,
 - ▶ el temps d'execució: major temps de cerca *vs* possibilitat de trobar cadenes més llargues.
- Comprimir és molt més lent que descomprimir.
- Cal codificar els tokens, o bé amb un codi fix, o bé combinat amb un codi de Huffman o una codificació aritmètica.

Storer, James A.; Szymanski, Thomas G. (October 1982). *Data Compression via Textual Substitution*. Journal of the ACM. 29 (4): 928–951.

[doi:10.1145/322344.322346](https://doi.org/10.1145/322344.322346)

LZSS és una variant de LZ77 que crea *tokens* amb dos elements en lloc de tres: *match-length* i *offset*. Si no es troba coincidència s'emet un literal (byte). Per distingir entre *tokens* i literals, cadascun està precedit per un sol bit (el *flag bit*). La mida de la finestra de cerca i la de la finestra avançada se selecciona tal que la mida total d'un *token* sigui de 16 bits (2 bytes). Per exemple, si la mida de la finestra de cerca és de 2 Kbytes (2^{11}), llavors la finestra avançada hauria de ser de 32 bytes de longitud (2^5).

Els *flags* es recopilen de 8 en 8 elements en bytes.

S'emet un byte que consta de 8 *flags*, seguit dels 8 elements (literals o *tokens*), que són d'1 o 2 bytes de longitud cadascun).

Exemple: patadecabraabracadabrapaaaaaaaaaataadecabr

[0, literal] o [1, (match-length, offset)]

||patadeca|braabracadabrapaaaaaaaaaataadecabr [0, 'p']
 |p|atadecab|raabracadabrapaaaaaaaaaataadecabr [0, 'a']
 |pa|tadecabr|aabracadabrapaaaaaaaaaataadecabr [0, 't']
 |pat|adecabra|abracadabrapaaaaaaaaaataadecabr
 [1, (1, 2)]
 |pata|decabraa|bracadabrapaaaaaaaaaataadecabr [0, 'd']
 |patad|ecabraab|racadabrapaaaaaaaaaataadecabr [0, 'e']
 |patade|cabraabr|ccadabrapaaaaaaaaaataadecabr [0, 'c']
 |patadec|abraab|racadabrapaaaaaaaaaataadecabr [1, (1, 4)]
 |patadeca|braabr|acadabrapaaaaaaaaaataadecabr [0, 'b']
 |patadecab|raabra|cadabrapaaaaaaaaaataadecabr [0, 'r']
 |patadecabr|aabrac|adabrapaaaaaaaaaataadecabr [1, (1, 3)]
 |patadecabra|abraca|dabrapaaaaaaaaaataadecabr [1, (4, 4)]
 |patadecabraabra|cadabr|apaaaaaaaaaataadecabr [1, (2, 9)]
 p|atadecabraabracad|abrap|aaaaaaaaaataadecabr [1, (1, 13)]
 pa|tadecabraabracad|abraba|aaaaaaaaaataadecabr
 [1, (4, 7)]
 patade|cabraabracadabra|paaaaa|aaaaaataadecabr [0, 'p']

Exemple: patadecabraabracadabrapaaaaaaaaaatadecabr

```
[...]
patadec|abraabracadabrap|aaaaaa|aaaaatadecabr [1, (2, 13)]
patadecab|raabracadabrapaa|aaaaaa|aaatadecabr [1, (6, 1)]
patadecabraabra|cadabrapaaaaaaaaa|aaatad|ecabr [1, (3, 1)]
patadecabraabracad|abrapaaaaaaaaaaaaa|tadeca|br [0, 't']
patadecabraabracada|brapaaaaaaaaaaaaat|adecab|r [1, (1, 2)]
patadecabraabracadab|rapaaaaaaaaaaaaata|decabr| [0, 'd']
patadecabraabracadabr|apaaaaaaaaaaaaatad|ecabr| [0, 'e']
patadecabraabracadabra|paaaaaaaaaaaaatade|cabr| [0, 'c']
[...]
```

Codi (match-length: 5 bits, offset: 11 bits):

```
00010001 'p' 'a' 't' 00000 000000000001 'd' 'e' 'c' 00000 000000000011
00111110 'b' 'r' 'r' 00000 000000000010 00011 000000000011 00001 00000001000 00000 00000001100 00011
000000000110 'p'
11101000 00001 00000001100 00101 000000000000 00011 000000000000 't' 00000 000000000001 'd' 'e' 'c'
```

```
00010001 01110000 01100001 01110100 000000000000000001 01100100 01100101 01100011 00000000000000011
00111110 01100010 01110010 000000000000000010 00011000000000011 00001000000001000 00000000000001100
00011000000000111 01110000 11101000 00001000000001100 00101000000000000 00011000000000000 01110100
000000000000000001 01100100 01100101 01100011
```

match-length, *offset* ≥ 1 : deso el valor menys 1.

Exemple:

0x11706174000164656300033e6272000218030808000c180770e8080c28001800740001646563...

Llegim el primer byte (*flags*): 0x11 = 00010001

Hem de llegir 10 bytes (8 + pes de Hamming del byte):

0x70(literal, byte) 0x61(lit.) 0x74(lit.) 0x0001(*token*) 0x64(lit.) 0x65(lit.) 0x63(lit.) 0x0003(*token*)
 'pat'(00000 000000000001)'dec'(00000 00000000011)
 'pat'(1, 2)'dec'(1, 4)
 'patadeca'

Llegim el següent byte (*flags*): 0x3e = 00111110

Hem de llegir 13 bytes: 62 72 0002 1803 0808 000c 1807 70

0x62(li.) 0x72(lit.) 0x0002(*token*) 0x1803(*token*) 0x0808(*token*) 0x000c(*token*) 0x1807(*token*) 0x70(lit.)
 'patadeca' 'br'(00000 00000000010)(00011 00000000011)(00001 00000001000)(00000 00000001100)(00011
 00000000111)'p'
 'patadecabr'(1, 3)(4, 4)(2, 9)(1, 13)(4, 7)'p'
 'patadecabr a abra ca d abra p'

Llegim el següent byte (*flags*): 0xe8 = 11101000

Hem de llegir 12 bytes: 080c 2800 1800 74 0001 64 65 63

0x080c(*token*) 0x2800 (*token*) 0x1800(*token*) 0x74(lit.) 0x0001(*token*) 0x64(lit.) 0x65(lit.) 0x63(lit.)
 'patadecabraabracadabrap'(00001 00000001100)(00101 00000000000)(00011 00000000000)'t'(00000
 00000000001)'dec'
 'patadecabraabracadabrap'(2, 13)(6, 1)(4, 1)'t'(1, 2)'dec'
 'patadecabraabracadabrap aa aaaaaa aaaa t a dec'

[...]

Si *match-length* és 1 és millor codificar un literal (17 bits vs. 9 bits).

DEFLATE Compressed Data Format Specification, RFC 1951

Usat per defecte a [GZIP file format specification, RFC 1952](#) i també a png

Combina LZSS amb codificació de Huffman

Treballa amb blocs encara que la finestra de cerca pot traspasar blocs. Els tres alfabets inicials:

- literal: bytes 0...255 més 256 EOB (end of block)
- match-length: 3-258
- offset: 1-32768 (2^{15})

es combinen en dos (veure RFC 1951 pàgina 11).

DEFLATE (i)

- Alfabet 1: literals + match-lengths: 286 símbols (si el match-length és < 3 , es guarden simplement els literals)

Extra			Extra			Extra		
Code	Bits	Length(s)	Code	Bits	Lengths	Code	Bits	Length(s)
257	0	3	267	1	15,16	277	4	67-82
258	0	4	268	1	17,18	278	4	83-98
259	0	5	269	2	19-22	279	4	99-114
260	0	6	270	2	23-26	280	4	115-130
261	0	7	271	2	27-30	281	5	131-162
262	0	8	272	2	31-34	282	5	163-194
263	0	9	273	3	35-42	283	5	195-226
264	0	10	274	3	43-50	284	5	227-257
265	1	11,12	275	3	51-58	285	0	258
266	1	13,14	276	3	59-66			

The extra bits should be interpreted as a machine integer stored with the most-significant bit first, e.g., bits 1110 represent the value 14.

DEFLATE (ii)

- offset: 1–32768 (2^{15}): 30 símbols + bits extres

Extra			Extra			Extra		
Code	Bits	Dist	Code	Bits	Dist	Code	Bits	Distance
----	----	----	----	----	-----	----	----	-----
0	0	1	10	4	33-48	20	9	1025-1536
1	0	2	11	4	49-64	21	9	1537-2048
2	0	3	12	5	65-96	22	10	2049-3072
3	0	4	13	5	97-128	23	10	3073-4096
4	1	5,6	14	6	129-192	24	11	4097-6144
5	1	7,8	15	6	193-256	25	11	6145-8192
6	2	9-12	16	7	257-384	26	12	8193-12288
7	2	13-16	17	7	385-512	27	12	12289-16384
8	3	17-24	18	8	513-768	28	13	16385-24576
9	3	25-32	19	8	769-1024	29	13	24577-32768

DEFLATE (iii)

Els diferents alfabetes es poden codificar amb:

- Huffman estàtic:

- ▶ literal + match-length: 286 símbols

Lit Value	Bits	Codes
-----	----	-----
0 - 143	8	00110000 through 10111111
144 - 255	9	110010000 through 111111111
256 - 279	7	0000000 through 0010111
280 - 287	8	11000000 through 11000111

- ▶ offset 30 símbols + bits extrems: 5 bits (+ els bits extrems)

- Huffman dinàmic: S'envien les longituds (que al seu torn es codifiquen amb Huffman preestablert)

Un mateix codificador pot combinar blocs amb Huffman estàtic i blocs amb Huffman dinàmic (i fins i tot blocs sense comprimir)

També es pot usar *Lazy Matching* consistent a codificar el millor resultat de la finestra actual i la finestra desplaçada una posició. En cas de ser millor la segona s'envia un caràcter i es desplaça la finestra una posició.

DEFLATE (iv)

Exemple: D, (4, 7), R, (5, 3), A, A, (16,1026)

Traduïm a símbols de l'alfabet corresponent:

68, 258_{+0bits}, 5_{+1bit}, 82, 259_{+0bits}, 2_{+0bits}, 65, 65, 267_{+1bit}, 20_{+9bits}

Codifiquem els símbols segons el Huffman estàtic:

01110100 68	0000010 258 _{+0bits}	00101 0 5 _{+1bit}	10000010 82	0000011 259 _{+0bit}	00010 2 _{+0bits}	01110001 65	01110001 65
0001011 1 267 _{+1bit}	10100 000000001 20 _{+9bit}						

El resultat final seria:

011101000000001000101010000010000001100010011100010111000100010111101000000000001

- Després d'un símbol del primer alfabet major que 256 (257–285) (match-length) sempre ve un símbol del segon alfabet (offset).
- Després d'un símbol del primer alfabet menor que 256 (0–255) sempre ve un altre símbol del primer alfabet.

Ziv, Jacob; Lempel, Abraham (September 1978). *Compression of Individual Sequences via Variable-Rate Coding*. IEEE Transactions on Information Theory. 24 (5): 530–536.
doi:10.1109/TIT.1978.1055934.

Exemple: mississippi mississippi river

```
-Diccionari: []
|mississippi mississippi river [0, 'm']
-Diccionari: ['m']
m|ississippi mississippi river [0, 'i']
-Diccionari: ['m', 'i']
mi|ssissippi mississippi river [0, 's']
-Diccionari: ['m', 'i', 's']
mis|issippi mississippi river [3, 'i']
-Diccionari: ['m', 'i', 's', 'si']
missi|ssippi mississippi river [3, 's']
-Diccionari: ['m', 'i', 's', 'si', 'ss']
mississ|iippi mississippi river [2, 'p']
-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip']
mississip|pi mississippi river [0, 'p']
-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p']
mississipp|i mississippi river [2, '']
-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i']
```

Exemple: mississippi mississippi river

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ']

mississippi |mississippi river [1, 'i']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi']

mississippi mi|ssissippi river [5, 'i']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi']

mississippi missi|ssissippi river [10, 'p']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip']

mississippi mississip|pi river [7, 'i']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip', 'pi']

mississippi mississippi| river [0, ' ']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip', 'pi', ' ']

mississippi mississippi |river [0, 'r']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip', 'pi', ' ', 'r']

mississippi mississippi r|iver [2, 'v']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip', 'pi', ' ', 'r', 'iv']

mississippi mississippi riv|er [0, 'e']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip', 'pi', ' ', 'r', 'iv', 'e']

mississippi mississippi rive|r [14, 'EOF']

-Diccionari: ['m', 'i', 's', 'si', 'ss', 'ip', 'p', 'i ', 'mi', 'ssi', 'ssip', 'pi', ' ', 'r', 'iv', 'e', 'r']

Codi: [0, 'm'], [0, 'i'], [0, 's'], [3, 'i'], [3, 's'], [2, 'p'], [0, 'p'], [2, ' '], [1, 'i'], [5, 'i'], [10, 'p'], [7, 'i'], [0, ' '], [0, 'r'], [2, 'v'], [0, 'e'], [14, 'EOF']

LZW (Welch 1984)

Welch, Terry (1984). *A Technique for High-Performance Data Compression*. Computer. 17 (6): 8–19. [doi:10.1109/MC.1984.1659158](https://doi.org/10.1109/MC.1984.1659158).

Elimina la necessitat d'enviar la lletra següent inicialitzant un diccionari, usualment amb els 256 bytes.

A l'hora de codificar anem acumulant lletres en una cadena P mentre està al diccionari.

En el moment en què $P||s$ no apareix al diccionari:

- i) Enviem l'índex de P .
- ii) Afegim $P||s$ al diccionari.
- iii) Inicialitzem $P = s$.

Exemple LZW (i)

Exemple: $a^1b^2a^3b^4a^5b^6a^7b^8b^9a^{10}$

Diccionari	Sortida
1. a	
2. b	
(1) llegim P=a,	
(2) llegim b, P=ab NO està al diccionari	1
3. ab	
P=b	
(3) llegim a, P=ba NO està al diccionari	2
4. ba	
P=a	
(4) llegim b, P=ab	
(5) llegim a, P=aba NO està al diccionari	3
5. aba	
P=a	
(6) llegim b, P=ab	
(7) llegim a, P=aba	
(8) llegim b, P=abab NO està al diccionari	5
6. abab	
P=b	

Exemple LZW (ii)

Exemple: $a^1b^2a^3b^4a^5b^6a^7b^8b^9a^{10}$

Diccionari

Sortida

1. a

2. b

3. ab

4. ba

5. aba

6. abab

P=b

(9) llegim b, P=bb NO està al diccionari

2

7. bb

P=b

(10) llegim a, P=ba

llegim EOF, P=baEOF NO està al diccionari

4

Codi: 1,2,3,5,2,4

Diccionari inicial: a, b

Exemple LZW (iii)

Exemple: Codi: 1,2,3,5,2,4

Diccionari inicial: a, b

Diccionari		Sortida acumulada
1. a		
2. b		
	llegim 1	a
3. a?	llegim 2	a b
3. ab		
4. b?	llegim 3	ab ab
4. ba		
5. ab?	llegim 5, NO està complet però sabem les primeres lletres Podem completar la 5a entrada del diccionari	abab ab?
5, aba		
	Completo la lletra del missatge que em faltava	abab aba
6. aba?		

Exemple LZW (iv)

Exemple: Codi: 1,2,3,5,2,4

Diccionari inicial: a, b

Diccionari

1. a
2. b
3. ab
4. ba
5. aba
6. aba? l·legim 2
6. abab
7. b? l·legim 4
7. bb

Sortida acumulada

abababa

abababa b

abababab ba

Diccionari dinàmic amb 2^9 entrades ($\Rightarrow$ 9 bits per codificar l'índex de l'entrada) de les quals 256 estan inicialitzades.

En omplir-se augmenta a 2^{10} entrades (10 bits per entrada), així successivament fins a 2^{16} entrades (16 bits per entrada).

A partir d'aquest moment es torna un diccionari estàtic mentre que la ràtio de compressió no disminueixi. En aquest instant es reinicialitza el diccionari.

En el cas de **GIF** en omplir-se el diccionari es reinicialitza.

GIF, png i les patents

Veure secció 3.29 (pàg 243) de [Data Compression: The Complete Reference](#) **David Solomon** Springer, 2004, 3rd ed.

(En aquesta referència també podeu trobar més detalls de tots els algorismes, i de gzip, gif, png, etc)